

Trabalho Prático 1: Análise Dinâmica de Carteiras de Ações

Bento Gabriel Araújo Matos

Matrícula: 2025049603

Departamento de Ciência da Computação
Universidade Federal de Minas Gerais (UFMG)
Belo Horizonte – MG – Brasil

bentogabrielmatos@gmail.com

1. Introdução

O problema abordado por este trabalho é o desenvolvimento de um sistema de recomendações financeiras para a empresa Jolambs. O sistema deve manter em tempo real diversas ações, cada uma com um histórico de cotações, e clientes com suas respectivas carteiras, além de possibilitar operações como compra, venda, nova cotação e consultas às melhores e piores ações possuídas por um cliente, com base em diferentes métricas que servem como critério para ordená-las e pontuá-las.

Nesse contexto, este trabalho tem como objetivo propor algoritmos e estruturas de dados que possibilitem realizar as operações do sistema de maneira eficiente, bem como analisar experimentalmente diferentes estratégias para alcançar esse objetivo. Para isso, foi desenvolvido um programa em C++ que lê entradas com os comandos, realiza as operações e imprime as saídas, tal como definido nas especificações.

Com isso, na seção 2, serão detalhadas as estruturas de dados, os tipos abstratos de dados (TADs) e os algoritmos aplicados, bem como as configurações utilizadas nos testes. Na seção 3, serão apresentadas as análises de complexidade de tempo e espaço dos algoritmos. Na seção 4, serão expostas as estratégias de robustez aplicadas. Na seção 5, serão descritas a realização e os resultados da análise experimental proposta. Finalmente, na seção 6, serão sumarizados o que foi feito e aprendido neste trabalho.

2. Método

O sistema proposto emprega uma arquitetura orientada a objetos em C++11, dividindo responsabilidades em componentes especializados para armazenar, processar e consultar dados de ações e clientes. A implementação utiliza loops para leitura da entrada baseada em linhas de comandos. A execução é dividida em duas etapas: primeiramente, são lidas as configurações de janela e métricas, seguidas pelo povoamento inicial de ações e clientes. Após essa etapa, o sistema processa as operações de compra, venda, atualização de cotação e consulta de carteira na ordem em que aparecem na entrada.

A seguir, detalham-se as estruturas de dados, tipos abstratos de dados e operações implementadas.

2.1. Estruturas de Dados

- **Vetores Circulares:** Utilizados para armazenar as cotações de cada ação, mantendo apenas as últimas entradas de forma eficiente, evitando realocações desnecessárias e permitindo

acesso rápido às informações mais recentes.

- **Listas Duplamente Encadeadas:** Empregadas para organizar os conjuntos de ações e clientes, bem como as carteiras individuais, facilitando inserções e remoções dinâmicas com acesso bidirecional.
- **Vetores de Ordenação:** Vetores cujos elementos são objetos do tipo **Tupla de Ordenação**. Essas estruturas permitem organizar referências às ações de acordo com critérios específicos, possibilitando tanto classificações globais (por métrica) quanto consultas personalizadas em carteiras.

2.2. Tipos Abstratos de Dados

- **Ação:** Representa uma unidade de investimento, encapsulando um identificador único, o histórico de cotações em um vetor circular e suas pontuações atribuídas nas ordenações globais. O TAD é capaz de calcular o valor de cada uma das métricas da respectiva ação.
- **Carteira:** Representa o conjunto de investimentos de um cliente, organizada como uma lista duplamente encadeada. Possui métodos para adição/remoção de ações e para gerar um vetor de ordenação baseado na soma ponderada das métricas.
- **Cliente:** Representa um usuário com um identificador único e sua carteira associada.
- **Conjunto de Ações:** Repositório centralizado de todas as ações disponíveis, organizadas em uma lista duplamente encadeada, além de possuir um vetor de ordenação para cada métrica disponível para manter a pontuação global atualizada.
- **Conjunto de Clientes:** Repositório centralizado de todos os clientes, organizados em uma lista duplamente encadeada alocada dinamicamente.
- **Tupla de Ordenação:** Estrutura auxiliar que armazena uma referência a uma ação e um valor numérico de interesse (como o resultado de uma métrica ou uma pontuação ponderada). É a unidade básica que compõe os vetores de ordenação, facilitando a aplicação de algoritmos de classificação.

2.3. Algoritmos

Para os algoritmos utilizados no sistema para realizar as operações, o momento em que eles serão executados depende da estratégia utilizada, as quais serão descritas na seção 5, porém, o funcionamento de cada um permanece o mesmo e são descritos a seguir.

- **QuickSort:** Uma versão adaptada do QuickSort recursivo que opera sobre vetores de tuplas de ordenação. A ordenação é decrescente pelo valor da métrica e, em caso de empate, o menor identificador de ação tem prioridade.
- **Inserção de Ação na Carteira (Compra):** Os ids da ação e do cliente são buscados em seus respectivos conjuntos; caso ambos sejam encontrados, uma referência à ação é adicionada à lista da carteira do cliente.
- **Remoção de Ação da Carteira (Venda):** O id do cliente é buscado no conjunto de clientes; caso seja encontrado, o id da ação é buscado em sua carteira para que a referência seja removida.
- **Nova Cotação:** O id da ação é buscado no conjunto de ações; caso encontrada, a nova cotação é inserida no vetor circular, sobrescrevendo a entrada mais antiga se a janela estiver cheia.
- **Cálculo de Métrica:** O valor de uma métrica é calculado via fórmula matemática utilizando as cotações presentes no vetor circular da ação.

- **Ordenação Global por Métrica:** Os vetores de ordenação global são atualizados com os valores atuais das métricas calculadas e reordenados via QuickSort para atualizar as pontuações de cada ação.
- **Ordenação Local da Carteira (Consulta):** Após validar o cliente, as pontuações das ações em sua carteira são ponderadas pelos pesos informados. Esses resultados populam um vetor de ordenação local que, após ser classificado pelo QuickSort, é utilizado para imprimir os resultados.

2.4. Configurações Utilizadas para Testes

As seguintes especificações foram utilizadas no desenvolvimento e testes do programa:

- **Sistema Operacional:** Ubuntu 24.04.4 LTS
- **Linguagem:** C++11
- **Compilador:** gcc 13.3.0
- **Processador:** AMD Ryzen 5 3400G 3.70 GHz
- **RAM disponível:** 8 GB

3. Análise de Complexidade

A avaliação de desempenho do sistema considera os custos computacionais das operações principais, expressos em notação assintótica. Assume-se que n representa o número total de ações, u o número de clientes, k o tamanho médio de uma carteira, m o número de métricas e w o tamanho da janela de cotações.

3.1. Complexidade de Tempo

- **Etapas de Configuração do Programa:** O processamento inicial das configurações e a população de dados ocorre em $O(n + u)$.
- **QuickSort:** A complexidade média do algoritmo é $O(x \log x)$ para uma entrada de tamanho x .
- **Inserção de Ação na Carteira (Compra):** A busca nos conjuntos resulta em $O(n + u)$.
- **Remoção de Ação da Carteira (Venda):** A busca no conjunto de clientes e na lista da carteira resulta em $O(u + k)$.
- **Nova Cotação:** A busca no conjunto de ações domina a operação, resultando em $O(n)$.
- **Cálculo de Métrica:** Como depende do percurso da janela de cotações, a complexidade é $O(w)$.
- **Ordenação Global por Métrica:** A atualização dos valores exige $O(n \cdot w)$ e a ordenação exige $O(n \log n)$, totalizando $O(n(w + \log n))$. Em cenários típicos onde w é pequeno e constante, a operação é dominada por $O(n \log n)$.
- **Ordenação Local da Carteira (Consulta):** O cálculo dos pesos para os elementos da carteira leva $O(k \cdot m \cdot w)$ e a ordenação local leva $O(k \log k)$, resultando em $O(k(m \cdot w + \log k))$.

3.2. Complexidade de Espaço

- **Armazenamento de Ações:** $O(n \cdot w)$ para o histórico de cotações. O uso de referências garante que não haja duplicidade de dados de ações entre conjuntos e carteiras.

- **Armazenamento de Vetores de Ordenação Global:** Requer $O(m \cdot n)$ para armazenar as tuplas de cada métrica.
- **Armazenamento de Clientes:** $O(u \cdot k)$ para a estrutura das carteiras.
- **Armazenamento Auxiliar (QuickSort):** A pilha de recursão demanda $O(\log n)$ para ordenação global e $O(\log k)$ para local. Na ordenação local, há ainda o uso de um vetor temporário de tamanho $O(k)$.

4. Estratégias de Robustez

Para garantir confiabilidade e segurança em um ambiente de processamento de dados financeiros, o sistema incorpora mecanismos de programação defensiva e tolerância a falhas, prevenindo comportamentos inesperados e facilitando a manutenção. Essas estratégias baseiam-se na validação sistemática das entradas e no controle explícito dos estados de execução.

Todas as linhas de entrada são verificadas durante o processamento. Cada linha é inicialmente decomposta em seus componentes básicos (comando e parâmetros), os quais são analisados individualmente. Linhas que violam alguma restrição são consideradas *defeituosas* e ignoradas, com a execução sendo retomada imediatamente na linha seguinte. Em casos de erros fatais, o programa é encerrado. As verificações realizadas são descritas a seguir.

4.1. Validação da Estrutura das Entradas

As linhas devem corresponder a um comando válido. Após a leitura, os parâmetros são extraídos e sua quantidade é comparada com a esperada para o comando identificado. A presença de parâmetros ausentes, excedentes ou de conteúdo residual na linha caracteriza uma entrada inválida. Essa verificação é realizada por meio de checagens diretas sobre os dados extraídos, garantindo que apenas entradas bem formadas avancem para as etapas seguintes. Isso evita que operações sejam executadas com informações inconsistentes, o que poderia gerar saídas incorretas.

4.2. Validação de Identificadores

Na definição de ações, clientes e nas consultas, esperam-se identificadores únicos e sequenciais. Durante a inserção, verifica-se se o identificador fornecido atende a essas propriedades. Já nas operações que fazem referência a elementos existentes, realiza-se uma verificação de pertencimento ao conjunto previamente construído. Essas validações são feitas por meio de consultas às estruturas de armazenamento em memória, impedindo acessos inválidos e garantindo a consistência dos dados manipulados.

4.3. Controle de Estados

Restrições sequenciais impedem modificações fora de ordem. O sistema mantém implicitamente o estado atual de execução (por exemplo, fase de configuração ou de processamento), e cada linha é validada também em função desse estado. Linhas de configuração não são aceitas após o encerramento da respectiva etapa, e operações não são permitidas durante a fase de configuração. Após a definição completa de ações e clientes, os conjuntos são considerados fechados e não aceitam novas inserções. Esse controle é realizado por meio de verificações condicionais associadas ao fluxo de execução, garantindo que as operações ocorram apenas em contextos válidos.

4.4. Validação de Valores

Os valores informados em cotações e nas definições de métricas para consultas devem pertencer aos intervalos válidos. Após a leitura, cada valor numérico é avaliado quanto ao seu domínio

permitido antes de ser utilizado em qualquer cálculo. Essa abordagem evita situações como divisões por zero ou resultados inválidos, assegurando a estabilidade das operações matemáticas realizadas pelo sistema.

4.5. Validação de Configuração

Durante a etapa de configuração, verifica-se a presença, a unicidade e a ordem das linhas obrigatórias (deve haver exatamente uma linha de configuração e pelo menos uma de população para ação e para cliente, nesta ordem). Essas verificações são realizadas ao longo da leitura, acompanhando a evolução do estado do sistema. Caso alguma inconsistência seja detectada, como ausência de elementos obrigatórios, repetição indevida ou valores inválidos de configuração, o programa é encerrado imediatamente. Essa decisão se justifica pelo fato de que uma configuração incorreta comprometeria todas as etapas subsequentes, tornando inválidos os resultados produzidos.

5. Análise Experimental

O objetivo desta análise é avaliar o compromisso entre a frequência de atualização das ordenações globais por métrica e a frequência de realização de consultas. Com base na análise de complexidade apresentada anteriormente, espera-se que o custo das ordenações globais, dominado por $O(n \log n)$, tenha impacto significativo no desempenho geral do sistema. Para isso, foram consideradas duas estratégias:

1. **Estratégia imediata:** as ordenações globais são atualizadas sempre que uma nova cotação impacta alguma métrica.
2. **Estratégia sob demanda:** as ordenações são reconstruídas apenas no momento em que uma consulta é realizada.

Dessa forma, levanta-se a hipótese de que a estratégia imediata tende a ser mais eficiente em cenários com alta frequência de consultas, enquanto a estratégia sob demanda é vantajosa quando predominam atualizações de cotações. A análise experimental tem como objetivo validar empiricamente essa hipótese.

5.1. Metodologia

Os experimentos consistiram na execução de múltiplos cenários controlados, nos quais diferentes parâmetros de entrada foram variados de forma isolada, mantendo os demais constantes. Essa abordagem permite observar diretamente o impacto de cada parâmetro no desempenho das estratégias.

Foram definidos três perfis experimentais, representando diferentes padrões de uso do sistema:

- **Zona de cruzamento:** frequências intermediárias entre cotações e consultas, evidenciando o ponto em que o desempenho relativo das estratégias se altera;
- **Cenário favorável à estratégia imediata:** alta frequência de consultas em relação às cotações;
- **Cenário favorável à estratégia sob demanda:** alta frequência de cotações em relação às consultas.

Cada cenário foi caracterizado pela proporção entre operações de cotação, operações de atualização de carteira (compra/venda) e consultas ao sistema, mantendo fixo o número total de operações.

Os parâmetros base utilizados nos experimentos foram:

- número de ações: 100;
- número de clientes: 20;
- tamanho médio das carteiras: 15;
- tamanho da janela das métricas: 5;
- grau de variação dos preços: 0.3;
- número total de operações: 3000.

A partir desses valores, cada parâmetro relevante foi variado individualmente, enquanto os demais permaneciam fixos. Para cada configuração, foram coletadas medidas como tempo total de execução, tempo médio por operação e custo médio por consulta e atualização. Os experimentos foram repetidos múltiplas vezes, e os resultados apresentados correspondem aos valores médios observados.

Os resultados foram organizados em gráficos, permitindo a comparação direta entre as estratégias em diferentes condições.

5.2. Resultados e Análise

Os resultados experimentais são apresentados a seguir, organizados de acordo com os diferentes cenários analisados.

5.2.1 Zona de Cruzamento

Impacto do Número de Ações A Figura 1 apresenta a razão de desempenho entre as estratégias e o tempo total de execução em função do número de ações.

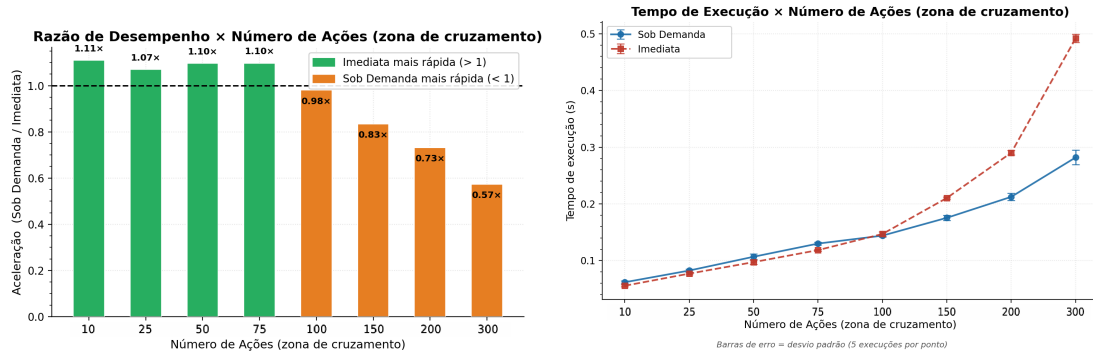


Figura 1: Zona de cruzamento: impacto do número de ações

Observa-se que, para valores menores de n , as estratégias apresentam desempenho semelhante. À medida que n cresce, o custo das ordenações globais se torna mais relevante, ampliando as diferenças entre as abordagens. Esse comportamento é consistente com a complexidade $O(n \log n)$, que passa a dominar o tempo total de execução conforme o tamanho da entrada aumenta.

Razão entre Cotações e Consultas A Figura 2 mostra a razão de desempenho entre as estratégias quando variada a frequência relativa de cotações e consultas.

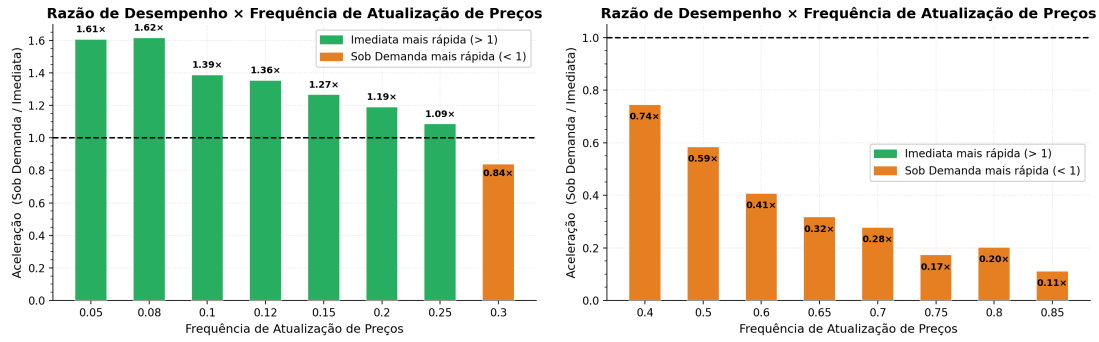


Figura 2: Zona de cruzamento: variação da razão entre cotações e consultas

Os gráficos evidenciam claramente o ponto de transição entre as estratégias. À medida que a proporção de consultas aumenta, a estratégia imediata passa a apresentar melhor desempenho, enquanto o oposto ocorre quando predominam cotações. A razão de desempenho permite identificar com precisão a região onde ocorre essa inversão, validando empiricamente a hipótese inicial.

5.2.2 Cenário Favorável à Estratégia Imediata

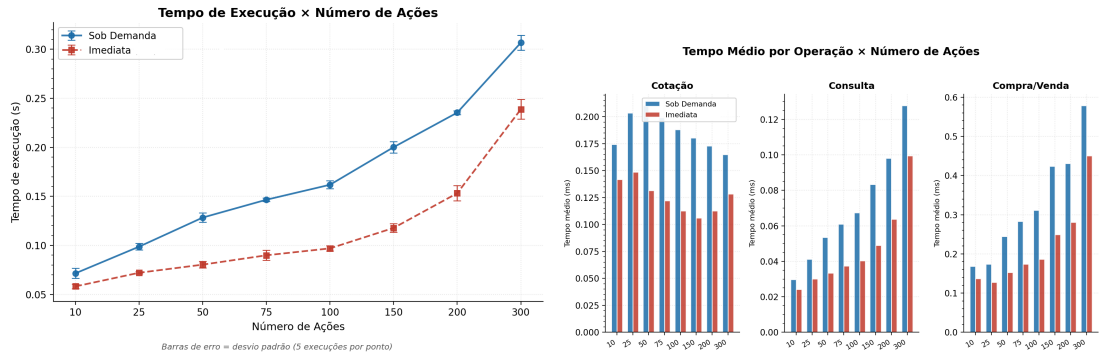


Figura 3: Cenário favorável à estratégia imediata: impacto do número de ações

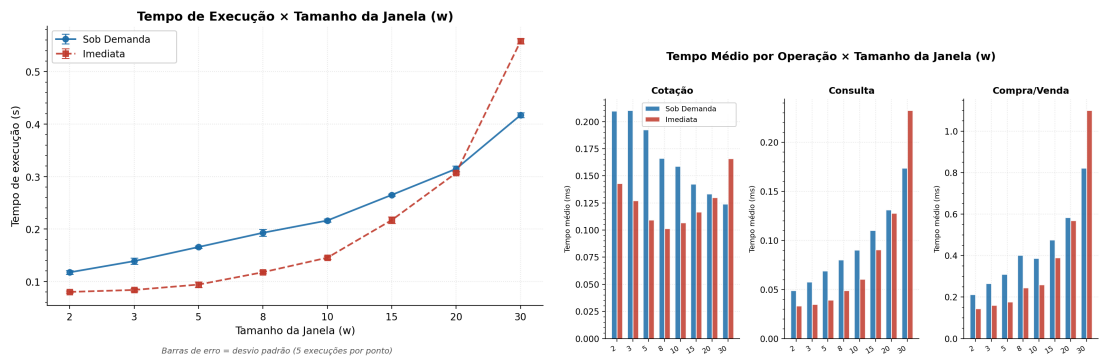


Figura 4: Cenário favorável à estratégia imediata: impacto do tamanho da janela

Nos cenários em que há predominância de consultas, a estratégia imediata apresenta desempenho superior de forma consistente. Esse comportamento se deve ao fato de que o custo de manutenção das ordenações globais, embora elevado ($O(n \log n)$), é distribuído ao longo das atu-

alizações. Assim, no momento da consulta, as estruturas já se encontram atualizadas, reduzindo significativamente o custo dessa operação.

Em contraste, na estratégia sob demanda, cada consulta implica a reconstrução das ordenações, acumulando repetidamente o custo de ordenação. Quando a frequência de consultas é elevada, esse custo passa a dominar o tempo total de execução.

5.2.3 Cenário Favorável à Estratégia Sob Demanda

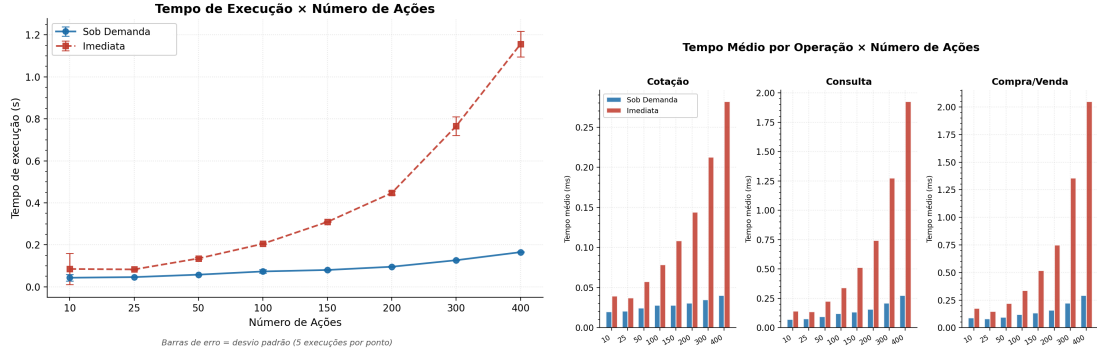


Figura 5: Cenário favorável à estratégia sob demanda: impacto do número de ações

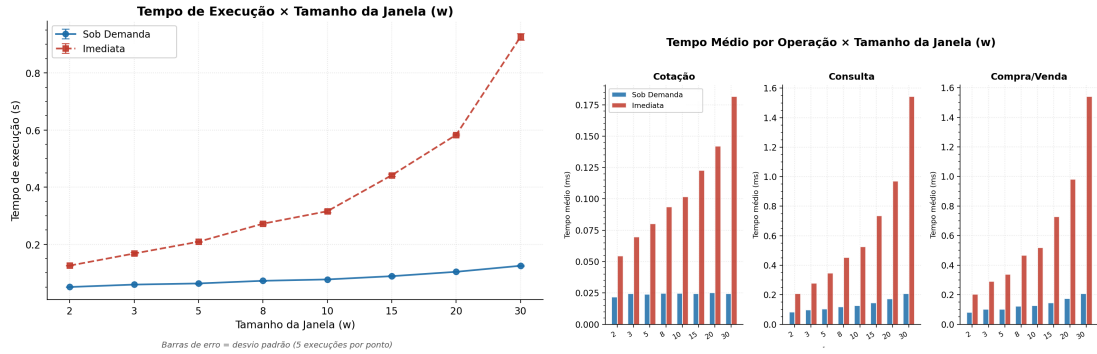


Figura 6: Cenário favorável à estratégia sob demanda: impacto do tamanho da janela

Nos cenários em que há predominância de cotações, a estratégia sob demanda apresenta melhor desempenho. Nessa abordagem, o custo das ordenações globais é evitado durante as atualizações e incorrido apenas quando necessário.

Como as ordenações possuem custo dominante de $O(n \log n)$, atualizá-las a cada nova cotação resulta em um acúmulo significativo de custo. Quando a frequência de consultas é baixa, muitas dessas atualizações não são aproveitadas. A estratégia sob demanda evita esse desperdício, realizando apenas as recomputações necessárias.

6. Conclusão

Este trabalho apresentou o desenvolvimento de um sistema de recomendações financeiras capaz de gerenciar ações, clientes e operações em tempo real, utilizando estruturas de dados eficientes e diferentes estratégias para manutenção de ordenações globais por métricas.

Ao longo do trabalho, foram exploradas soluções baseadas em vetores circulares, listas encadeadas e algoritmos de ordenação, permitindo a implementação eficiente das operações propostas.

A análise de complexidade evidenciou que o custo dominante do sistema está associado às ordenações globais, influenciando diretamente o desempenho das estratégias adotadas.

A partir da análise experimental, foi possível compreender, na prática, o impacto da distribuição das operações no desempenho do sistema. Observou-se que a estratégia imediata é mais eficiente em cenários com alta frequência de consultas, enquanto a estratégia sob demanda apresenta melhor desempenho quando predominam atualizações de cotações. Os resultados evidenciam que não existe uma estratégia universalmente superior, sendo a escolha diretamente dependente do perfil de utilização do sistema.

De forma geral, o trabalho permitiu consolidar conceitos fundamentais de estruturas de dados, análise de algoritmos e avaliação experimental, evidenciando a importância de alinhar decisões de projeto tanto à análise teórica quanto ao comportamento observado na prática.

Referências

1. Anisio Lacerda, Giesele L. Pappa, Marcio Santos e Wagner Meira Jr. (2026). *Especificação do Trabalho Prático 1: Análise Dinâmica de Carteiras de Ações*. Disponibilizado via Moodle. Departamento de Ciência da Computação, UFMG.
2. Anisio Lacerda, Giesele L. Pappa, Marcio Santos e Wagner Meira Jr. (2026). *Slides virtuais da disciplina de Estruturas de Dados*. Disponibilizado via Moodle. Departamento de Ciência da Computação, UFMG.